FOCUSCHAIN LABS · ENGINEERING

Multi-Tenant SaaS Architecture & GCP Runbook

How the FocusChain CRM serves FocusChain Labs and SN Realtors from one codebase — the design, why it works, and exactly what to click and run on Google Cloud to take it live.

Google Sign-In · organization_id isolation · WhatsApp Embedded Signup
Daily AI batch (~Rs 2,000/mo) · Cloud Run · Cloud SQL · CI/CD

Contents

PART I

The System

What the system is, the shape of it, and the decisions that make it cheap and safe.

PART I · 1

Executive summary

FocusChain CRM is one application that serves two independent companies — FocusChain Labs and SN Realtors — from a single codebase and a single set of cloud resources. Each company (a "tenant" or "organisation") sees only its own leads, conversations and WhatsApp numbers, with its own branding, behind Google Sign-In.

The system is built on three ideas that make it cheap, safe and easy to operate:

- **Shared multi-tenancy.** Every row of business data carries an organization_id. One shared database, partitioned in software — not a separate database per customer. This is the cheapest model that still guarantees isolation.
- **Identity = tenant.** A user's tenant is derived from their verified Google email domain at sign-in — never from anything they can type — and every query is scoped to it. Unknown domains are refused.
- **Batched AI.** Inbound WhatsApp messages are stored cheaply and enriched by one AI call per active contact, once a day — keeping the AI bill near Rs 2,000/month even at 500+ leads per tenant.

Everything runs on Google Cloud Run (scale-to-zero containers) with Cloud SQL (PostgreSQL) for storage, deploys itself from GitHub on every push to main, and is observable per-tenant through structured logs. Part III is a literal click-and-command runbook to take it from an empty Google Cloud project to production.

> **Who this document is for**
> You (the operator) — to understand the system deeply and to stand it up on GCP yourself. No prior Google Cloud experience is assumed in Part III: every step says where to go in the console and the exact command to run.

2 Architecture at a glance

Two public entry points (the web app and the WhatsApp webhook), one shared database, and a scheduled job for the daily AI work. Both companies are served by the same boxes; the organization_id on every row keeps their data apart.

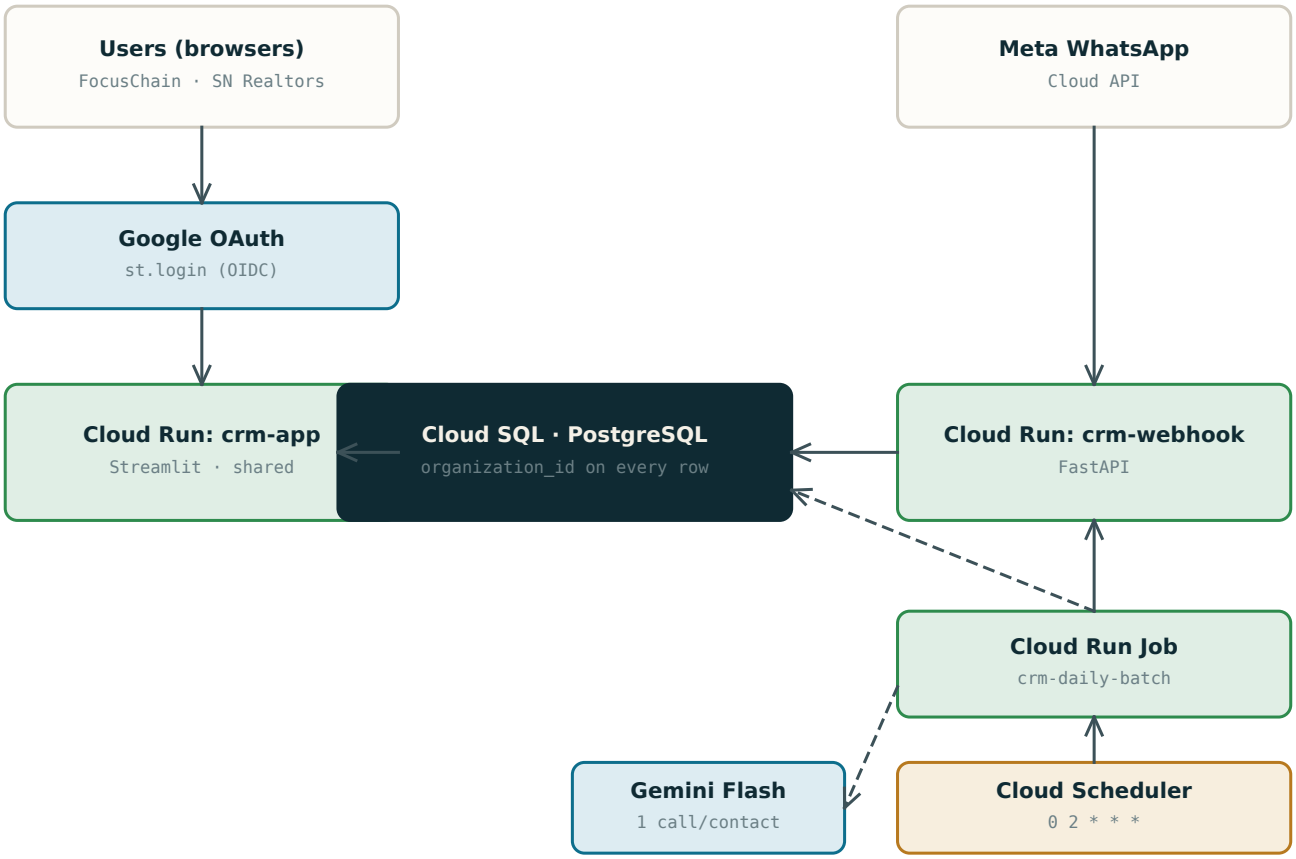

Figure 1 — Deployment topology. Solid = request/data path · dashed = the daily batch.

3 Why this architecture (key decisions)

Each choice below trades complexity for either cost or safety. The recurring theme: keep one of everything, and separate tenants in software.

Decision	What we chose	Why
Tenancy model	Shared tables + organization_id	One DB and one app for both tenants. A database-per-tenant model would roughly double cost and ops for two customers; isolation is still guaranteed by scoping every query.
Tenant identity	Google email domain -> org	No passwords to manage; the domain is verified by Google. A user can never pick another tenant — it is derived server-side.
Compute	Cloud Run (scale to zero)	Pay only when used; both services idle to zero. Fits a low monthly budget and needs no servers to patch.
WhatsApp onboarding	Meta Embedded Signup	The official, compliant 'scan a QR' flow. Unofficial libraries risk the business number being banned and need an always-on server.
AI processing	Daily batch, 1 call/contact	An LLM call per message is unaffordable at 500+ leads. Batching to one call per active contact per day holds the bill near Rs
Deploys	GitHub Actions -> Cloud Run	Push to main and it ships. No manual deploys; identity via Workload Identity Federation means no long-lived keys in GitHub.

PART II

How It Works

The mechanisms — tenancy, identity, WhatsApp, AI cost, delivery and observability — and why each is built the way it is.

PART II · 4

Multi-tenancy: the organization_id model

A tenant is a customer company. The whole system rests on one rule: every row of business data belongs to exactly one tenant, named by its organization_id, and no query ever returns rows from another tenant.

The data model

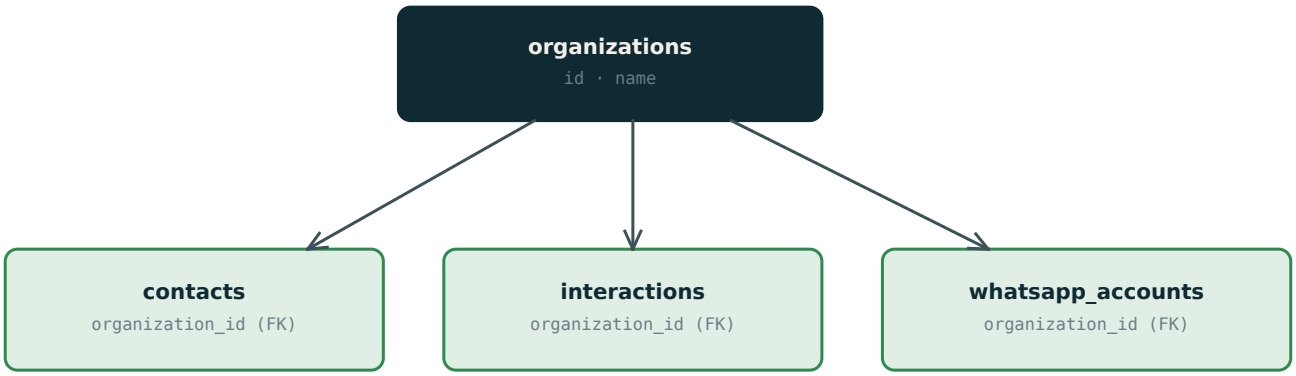

Figure 2 — One row per tenant in organizations; every owned row points back to it.

All reads filter by organization_id; all writes stamp it. The Streamlit UI, the WhatsApp webhook, the mobile REST API and the daily batch all go through one storage layer (utils/crm_store_postgres.py) that takes organization_id as an explicit argument — it is never read from a request body or a webhook payload, both of which an attacker could forge.

Three isolation guarantees

- **No cross-tenant overwrite.** A globally-unique primary key (a contact id, a phone_number_id) that collides across tenants cannot be overwritten — the UPDATE carries a WHERE organization_id = ... guard, so a write to the wrong tenant is a no-op, not corruption.
- **Scoped deletes.** Replacing a tenant's contact list computes its delete-set only within that tenant, so a save can never delete another tenant's rows.
- **Server-side routing.** Inbound WhatsApp traffic is mapped to a tenant by looking up our own whatsapp_accounts table (phone_number_id -> organization_id), never by trusting the message payload.

Fail-closed on the wrong backend

Isolation only holds on Cloud SQL (where rows carry organization_id). If sign-in is enabled but Cloud SQL isn't configured, the app refuses to load rather than fall back to a shared file/Supabase store that has no per-tenant scoping. Better a locked door than a leak.

Deferred: PostgreSQL Row-Level Security (RLS)

Today isolation is enforced in the application layer (every query is scoped). A future hardening pass can add database-enforced RLS policies so isolation holds even if a query forgets its filter. Documented as a follow-up, not a blocker for two trusted tenants.

PART II · 5

Authentication & tenant resolution

Sign-in uses Streamlit's native OpenID Connect with Google. The clever part isn't the login — it's turning a verified identity into a tenant and a data scope, with no way for the user to influence which tenant they land in.

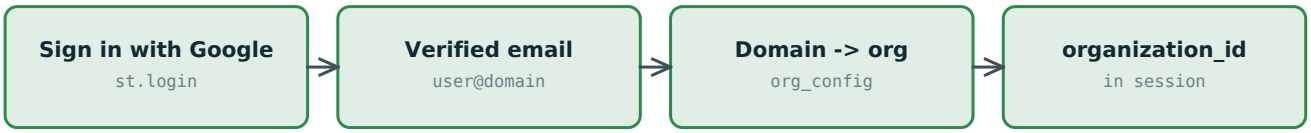

Figure 3 — From Google identity to a scoped tenant in four steps.

How the domain maps to a tenant

utils/org_config.py is the registry of tenants: each has an id, display name, branding and a list of email domains. resolve_org_for_email() matches the verified domain to a tenant; an unmapped domain returns None and the user is shown an access-denied screen. The mapping is overridable in production via the ORG_CONFIG or ORG_EMAIL_DOMAINS secrets, so you can add a domain without a code change.

```
ORG_EMAIL_DOMAINS = '{"focuschainlabs.com":"focuschainlabs",
                      "snrealtors.in":"sn_realtors"}'
```

Branding follows the tenant

Once the tenant is known, the same UI renders that tenant's name, tagline and accent — FocusChain Labs or SN Realtors — so each company sees "their" app. The manual org-switcher that exists for local development is hidden under real auth, so a signed-in user cannot switch tenants by hand.

> **Local development needs no Google client**
> If the [auth] secrets aren't present, the app runs unauthenticated in single-tenant 'dev mode' (set DEV_ORG_ID to pick which tenant to preview). Production turns auth on simply by adding the [auth] section.

PART II · 6

WhatsApp: Embedded Signup & inbound routing

Each tenant connects its own WhatsApp number(s) from inside the CRM using Meta's official Embedded Signup — the compliant 'scan a QR in Meta's popup' flow. The hard problem is binding that connection to the right tenant without trusting the browser.

The connect flow (and its security)

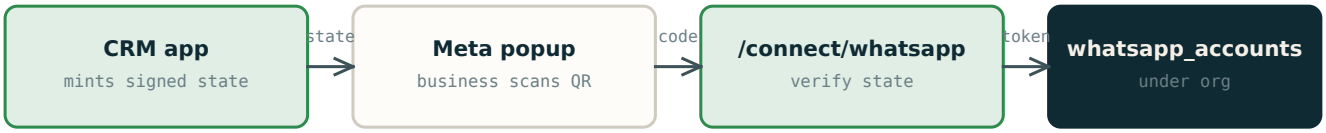

Figure 4 — The signed state carries the tenant end-to-end; the browser can't forge it.

The app (which knows the signed-in tenant) mints a short-lived HMAC-signed token that encodes the organization_id. The browser passes it through Meta's popup to the webhook's /connect/whatsapp endpoint, which verifies the signature, exchanges Meta's authorization code for a business token, asks Meta for the authoritative phone number, and stores it under that tenant. Because the tenant comes from the signed token (not a browser field), one tenant can never attach a number to another.

Inbound messages route themselves

When a customer messages a connected number, Meta calls the webhook with the receiving phone_number_id. The webhook looks that up in whatsapp_accounts to find the organization_id, then finds-or-creates the lead and stores the message — all under the right tenant, with no payload trust. Retried deliveries are de-duplicated by Meta's message id.

PART II · 7

The AI cost engine: daily batch

Reading and updating a CRM record from free-text messages needs an LLM. Doing that per message, in the webhook, is the obvious design — and it's unaffordable at 500+ leads per tenant. The fix is to separate cheap storage from expensive understanding.

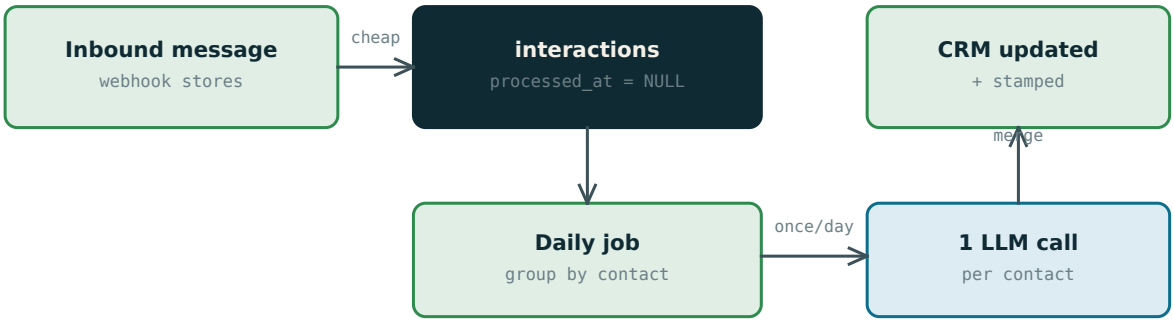

Figure 5 — Store cheaply now; understand in one batched call per active contact, later.

INBOUND_LLM_MODE=batch tells the webhook to store messages without an LLM call (processed_at stays NULL). Once a day, a Cloud Run Job groups the pending messages by contact and makes a single Gemini Flash call per active contact, merges the result into the record (filling only empty fields and accumulating notes, so it never clobbers what a human typed), and stamps the messages processed. A per-tenant cap (DAILY_LLM_BUDGET) bounds spend; anything over the cap simply waits for tomorrow. Stamping makes re-runs idempotent and cheap.

Why it fits ~Rs 2,000/month/tenant

Item	Estimate
Model	Gemini 2.5 Flash
Tokens per call	~1.5k in + ~0.3k out
Cost per call	~ Rs 0.10 – 0.12
Daily cap (DAILY_LLM_BUDGET)	500 calls / tenant / day
Worst case	500 x 30 x Rs 0.11 = ~ Rs 1,650 / month / tenant
Realistic	far fewer contacts active per day -> well under the cap

PART II · 8

CI/CD & deployment topology

Pushing to the main branch on GitHub builds and deploys the affected service to Cloud Run automatically. There are no long-lived cloud keys stored in GitHub: Workload Identity Federation lets the GitHub Action prove its identity to Google and assume a deploy service account just for that run.

- **App.** .github/workflows/deploy-app.yml builds Dockerfile.streamlit and deploys the crm-app service.
- **Webhook + job.** .github/workflows/deploy-webhook.yml builds the root Dockerfile, deploys crm-webhook, and updates the crm-daily-batch job image in lock-step.
- **Labels.** Every deploy attaches labels (app, component, env, tenant-model) so resources and billing are filterable.

One image, two runtimes

The webhook image also contains scripts/, so the daily batch job reuses the exact same image — it just overrides the command. One build, always in sync.

PART II · 9

Observability per tenant

In a shared deployment there is no per-tenant Cloud Run service to look at, so per-tenant visibility comes from the logs. The app, webhook and batch emit single-line JSON (utils/obs.py) that Cloud Logging parses into structured fields — including organization_id.

```
jsonPayload.event="inbound_message"
jsonPayload.organization_id="focuschainlabs"
```

From those logs you build log-based metrics with an organization_id label: inbound volume per tenant, and daily LLM calls per tenant (your cost watchdog against the budget). One dashboard then breaks every chart out by tenant. Resource labels drive the billing breakdown (Billing -> Reports -> group by label). Full queries are in docs/OBSERVABILITY.md.

PART III

GCP Production Runbook

From an empty Google Cloud project to a live, auto-deploying, two-tenant system. Each step says where to go in the console and the exact command. Commands assume the gcloud CLI; you can do most of it in the console UI too. Set these once and reuse them:

```
export PROJECT_ID=focuschain-crm           # your GCP project id
export REGION=asia-south1                  # Mumbai
export REPO=focuschain                     # Artifact Registry repo
gcloud config set project "$PROJECT_ID"
```

Step 0 — Prerequisites

CONSOLE: console.cloud.google.com

You need: a Google account with billing, the GitHub repo, and a phone with the WhatsApp Business app. Install the gcloud CLI locally (or use Cloud Shell — the >_ icon, top-right of the console — which has gcloud pre-installed).

```
gcloud auth login
gcloud components update
```

Step 1 — Create the project & enable billing

CONSOLE: Console -> top project dropdown -> New Project; then Billing -> Link a billing account

1. Create a project (note its Project ID, e.g. focuschain-crm).
2. Billing -> link a billing account to it.
3. Set it as your active project with the command below.

```
gcloud projects create "$PROJECT_ID"   # or pick an existing one
gcloud config set project "$PROJECT_ID"
```

Step 2 — Enable the APIs

CONSOLE: APIs & Services -> Enable APIs and Services

Turn on every API the system uses in one command.

```
gcloud services enable \
  run.googleapis.com sqladmin.googleapis.com \
  artifactregistry.googleapis.com cloudbuild.googleapis.com \
  cloudscheduler.googleapis.com secretmanager.googleapis.com \
  iamcredentials.googleapis.com logging.googleapis.com
```

Step 3 — Create the Artifact Registry repo

CONSOLE: Artifact Registry -> Create Repository (Format: Docker, Region: asia-south1)

This is where your container images live. The workflows push to repo name 'focuschain'.

```
gcloud artifacts repositories create "$REPO" \
  --repository-format=docker --location="$REGION" \
  --description="FocusChain CRM images"
```

Step 4 — Provision Cloud SQL (PostgreSQL)

CONSOLE: SQL -> Create Instance -> PostgreSQL

1. Create a small Postgres 15 instance (db-g1-small is fine to start).
2. Create a database named focuschain and a user focuschain_user.
3. Note the INSTANCE CONNECTION NAME (looks like project:region:instance) — Cloud Run uses it.
4. Apply the schema (run the file in db/schema_cloudsql.sql).

```
gcloud sql instances create focuschain-db \
  --database-version=POSTGRES_15 --tier=db-g1-small --region="$REGION"
gcloud sql databases create focuschain --instance=focuschain-db
gcloud sql users create focuschain_user --instance=focuschain-db --password='CHOOSE_A_STRONG_PW'
# apply schema (Cloud SQL Studio in the console, or via the proxy):
psql "$DATABASE_URL" -f db/schema_cloudsql.sql
```

Connection name
Save it: export CLOUD_SQL_CONNECTION_NAME="$PROJECT_ID:$REGION:focuschain-db". This is what links Cloud Run to the database.

Step 5 — Store secrets

CONSOLE: Security -> Secret Manager -> Create Secret

Create one secret per sensitive value (DB password, API keys, OAuth client secret, Meta app secret, WA_CONNECT_SECRET). You'll reference them from Cloud Run with --set-secrets. Generate the shared connect secret:

```
python -c "import secrets; print(secrets.token_hex(32))"   # WA_CONNECT_SECRET
printf '%s' "YOUR_DB_PASSWORD" | gcloud secrets create DB_PASSWORD --data-file=-
```

Step 6 — Create the Google OAuth client (for sign-in)

`CONSOLE: APIs & Services -> Credentials -> Create Credentials -> OAuth client ID -> Web application`

1. Configure the OAuth consent screen (Internal if both companies use Google Workspace you control; otherwise External).
2. Create a Web application client.
3. Add an Authorized redirect URI: https://YOUR-APP-URL/oauth2callback (you'll get the real URL after Step 9 — come back and add it).
4. Copy the Client ID and Client Secret into the app's [auth] secrets.

> **The [auth] block**
> In the app's Streamlit secrets set: redirect_uri, a random cookie_secret, and [auth.google] client_id / client_secret / server_metadata_url. Template is in .streamlit/secrets.example.toml.

Step 7 — Set up the Meta app (WhatsApp Embedded Signup)

`CONSOLE: developers.facebook.com -> My Apps -> Create App (Business) -> add WhatsApp`

1. Create a Business app; add the WhatsApp product.
2. Set up Embedded Signup (Configurations) and note the Configuration ID.
3. From App settings copy App ID and App Secret.
4. Point the WhatsApp webhook to https://YOUR-WEBHOOK-URL/webhook with your WHATSAPP_VERIFY_TOKEN; subscribe to the messages field.

> **Maps to env vars**
> META_APP_ID, META_APP_SECRET, META_CONFIG_ID, WA_CONNECT_SECRET, WEBHOOK_PUBLIC_URL, APP_PUBLIC_URL. See .streamlit/secrets.example.toml.

Step 8 — First deploy: the webhook (manual, once)

`CONSOLE: Cloud Run -> the service appears after this command`

Build, push and deploy the webhook image once by hand so the service exists (CI takes over afterwards). Wire in Cloud SQL and its secrets.

```
IMG="$REGION-docker.pkg.dev/$PROJECT_ID/$REPO/webhook:bootstrap"
gcloud builds submit --tag "$IMG" .
gcloud run deploy crm-webhook --image "$IMG" --region "$REGION" \
  --allow-unauthenticated \
  --add-cloudsql-instances "$CLOUD_SQL_CONNECTION_NAME" \
  --set-env-vars INBOUND_LLM_MODE=batch,CLOUD_SQL_CONNECTION_NAME="$CLOUD_SQL_CONNECTION_NAME" \
  --set-secrets DB_PASSWORD=DB_PASSWORD:latest,WHATSAPP_ACCESS_TOKEN=WA_TOKEN:latest
```

Step 9 — First deploy: the app (manual, once)

`CONSOLE: Cloud Run -> crm-app`

Same idea for the Streamlit app. Its URL is what you put in the Google OAuth redirect URI (Step 6) and in APP_PUBLIC_URL / WEBHOOK_PUBLIC_URL.

```
IMG="$REGION-docker.pkg.dev/$PROJECT_ID/$REPO/app:bootstrap"
gcloud builds submit --tag "$IMG" -f Dockerfile.streamlit .
gcloud run deploy crm-app --image "$IMG" --region "$REGION" \
  --allow-unauthenticated \
  --add-cloudsql-instances "$CLOUD_SQL_CONNECTION_NAME" \
  --set-env-vars CLOUD_SQL_CONNECTION_NAME="$CLOUD_SQL_CONNECTION_NAME"
```

Now finish Step 6
Copy the crm-app URL, add /oauth2callback to the Google OAuth redirect URIs, and set redirect_uri to match. Sign-in won't work until these line up exactly.

Step 10 — Turn on auto-deploy (Workload Identity Federation)

`CONSOLE: IAM & Admin -> Workload Identity Federation -> Create Pool`

1. Create a Workload Identity Pool + an OIDC provider for GitHub (issuer https://token.actions.githubusercontent.com).
2. Create a deploy service account; grant it run.admin, iam.serviceAccountUser, artifactregistry.writer, cloudbuild.builds.editor.
3. Allow your GitHub repo to impersonate it (attribute on repository).
4. Add four GitHub repo secrets so the workflows can authenticate.

```
# GitHub -> repo -> Settings -> Secrets and variables -> Actions:
GCP_PROJECT_ID=...        GCP_SERVICE_ACCOUNT=deployer@PROJECT.iam.gserviceaccount.com
GCP_WORKLOAD_IDENTITY_PROVIDER=projects/NUM/locations/global/workloadIdentityPools/POOL/providers/PROVIDER
```

After this
Every push to main runs the workflows in .github/workflows and deploys automatically — no keys stored.

Step 11 — Schedule the daily AI batch

`CONSOLE: Cloud Run -> Jobs -> Create Job; then Cloud Scheduler`

1. Create the crm-daily-batch job from the webhook image, overriding the command. Give it Cloud SQL + the same env.
2. Create a Scheduler trigger to run it once a day.
3. Confirm the webhook has INBOUND_LLM_MODE=batch so messages defer to this job.

```
gcloud run jobs create crm-daily-batch --image "$IMG_WEBHOOK" --region "$REGION" \
  --command python --args scripts/process_inbound_daily.py,--all \
  --add-cloudsql-instances "$CLOUD_SQL_CONNECTION_NAME" \
  --set-env-vars INBOUND_LLM_MODE=batch,DAILY_LLM_BUDGET=500
gcloud scheduler jobs create http crm-daily-batch-trigger \
  --schedule "0 2 * * *" --location "$REGION" \
  --uri "https://$REGION-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/$PROJECT_ID/jobs/crm-daily
  --http-method POST --oauth-service-account-email "$RUN_INVOKER_SA"
```

Step 12 — Labels, metrics & a per-tenant dashboard

CONSOLE: Logging -> Log-based Metrics; Monitoring -> Dashboards

1. Deploys already label resources; filter the Cloud Run list with labels.app:focuschain-crm.
2. Create a log-based counter on jsonPayload.event="inbound_message" with a label organization_id.
3. Create a distribution metric on jsonPayload.event="daily_batch" value jsonPayload.llm_calls, label organization_id — your cost watchdog.
4. Build a dashboard grouped by organization_id. Full queries: docs/OBSERVABILITY.md.

Step 13 — Custom domain (optional)

CONSOLE: Cloud Run -> Manage Custom Domains

Map a domain (e.g. app.focuschainlabs.com) to crm-app, update the DNS records it shows you, then update the Google OAuth redirect URI and APP_PUBLIC_URL to the new domain.

Step 14 — Assign existing data to a tenant

CONSOLE: Cloud SQL Studio (or psql)

Leads created before multi-tenancy carry organization_id='default' and are invisible to the new tenants. Assign them once.

```
UPDATE contacts     SET organization_id='focuschainlabs' WHERE organization_id='default';
UPDATE interactions SET organization_id='focuschainlabs' WHERE organization_id='default';
```

Step 15 — Go-live checklist

- ✓ Cloud SQL schema applied (db/schema_cloudsql.sql); existing data assigned.
- ✓ crm-app and crm-webhook deploy green from GitHub on push to main.
- ✓ Google sign-in works; a focuschainlabs user sees FocusChain branding, an sn_realtors user sees SN Realtors — and neither sees the other's leads.
- ✓ A test WhatsApp message to a connected number lands on the right tenant.
- ✓ crm-daily-batch runs on schedule; daily_batch logs show calls under the cap.
- ✓ Per-tenant metrics visible; a budget alert is set.

REFERENCE

Appendix A — Environment variables

Variable	Where	Purpose
CLOUD_SQL_CONNECTION_NAME	app, webhook, job	Links Cloud Run to Cloud SQL (project:region:instance).
DATABASE_URL	local dev	Postgres URL when not on Cloud Run.
[auth] + auth.google	app	Google OIDC sign-in (redirect_uri, cookie_secret, client id/secret).
ORG_CONFIG / ORG_EMAIL_DOMAINS	app	Tenant registry & email-domain -> org mapping.
API_KEYS	webhook	Bearer token -> org for the mobile REST API.
META_APP_ID / META_APP_SECRET	app / webhook	Meta app for Embedded Signup + token exchange.
META_CONFIG_ID	app	Embedded Signup configuration id.
WA_CONNECT_SECRET	app + webhook	Shared HMAC secret binding a connect to a tenant.
WEBHOOK_PUBLIC_URL / APP_PUBLIC_URL	app / webhook	Public URLs (popup target + CORS allow-list).
INBOUND_LLM_MODE	webhook	'batch' defers per-message LLM to the daily job.
DAILY_LLM_BUDGET	job	Max LLM calls per tenant per run (default 500).

REFERENCE

Appendix B — Troubleshooting

"Almost there" config screen after login

Auth is on but Cloud SQL isn't reachable. Check CLOUD_SQL_CONNECTION_NAME and that the service has --add-cloudsql-instances.

Sign-in loops or 'redirect_uri mismatch'

The Google OAuth redirect URI must exactly equal redirect_uri in secrets, including https and /oauth2callback.

A tenant's CRM is empty

Likely the pre-existing data is still under organization_id='default' — run the Step 14 assignment.

WhatsApp messages don't appear

Confirm the number is in whatsapp_accounts under that tenant, the webhook URL/verify token match Meta, and the messages field is subscribed.

Daily batch did nothing

It only processes unprocessed inbound rows. If the webhook is in realtime mode, messages are stamped processed on arrival; set INBOUND_LLM_MODE=batch.

AI bill creeping up

Watch the daily_batch llm_calls metric; lower DAILY_LLM_BUDGET or confirm batch mode is on.